

Portfolio Website — Design and Build Report

Niloufar Modirshanehchi

Live site: <https://persianspock.github.io> · Repository:
github.com/PersianSpock/PersianSpock.github.io

1. Introduction

This report documents the design and construction of my personal portfolio website. The site is a single, hand-authored static page that introduces my background, offers my CV for download, and presents one machine-learning project as an in-depth case study. It is published with GitHub Pages and the complete source — including this report and the LaTeX source of my CV — lives in the repository linked above.

The guiding principle behind every decision was the same one I bring to data work: prefer things that can be inspected and verified. That shows up in small ways throughout — the CV is kept as LaTeX source rather than only a PDF, the project’s reported metrics are attributed to their source, and the performance and accessibility claims in this report are backed by measurements I ran rather than assertions.

2. Website structure

The page is built from five semantic regions, in reading order:

- **Header / navigation.** A sticky top bar with a monogram and anchor links (About, Project, Toolkit, CV, Contact). It stays in view so navigation is one click from anywhere on the page.
- **Hero.** A short positioning statement paired with the site’s signature element: a small ROC (receiver operating characteristic) curve rendered in inline SVG that animates — drawing itself from the origin — on load. The ROC curve was chosen deliberately: it is the canonical picture of a classifier separating signal from chance, which both ties to the featured project and states the site’s theme (measurement) without a word of explanation.
- **About.** A two-column block: a large lead sentence on my current role and a supporting paragraph on prior data-science and NLP work.
- **Featured project.** The churn-prediction case study: a one-line problem framing, a two-column “Approach / Results” layout, animated result bars, and three call-to-action links (source code, Colab notebook, full case study).
- **Toolkit.** A compact set of labelled “chips” summarising the technologies I work with, drawn directly from my CV.
- **CV band and footer.** A download button for the PDF with a note that the CV is LaTeX-authored, followed by a contact footer (email, GitHub, LinkedIn).

Each region maps to an HTML5 landmark (**header**, **main**, **section**, **footer**, **nav**) so the document is navigable by structure, not just by sight.

3. Repository structure

The repository mirrors the layout below. The directory tree is intentionally flat and predictable so a reviewer can find any artefact without guessing.

```

PersianSpock.github.io/
|-- index.html           # portfolio homepage (semantic HTML)
|-- css/style.css        # single stylesheet: tokens + responsive rules
|-- assets/favicon.svg   # site favicon
|-- cv/
|   |-- cv.tex           # CV - LaTeX source
|   |-- cv.pdf           # CV - compiled PDF (linked from the site)
|-- projects/
|   |-- churn-prediction/
|       |-- README.md    # case study; links to source repo + Colab
|-- report/
|   |-- report.tex       # this report - LaTeX source
|   |-- report.pdf       # this report - compiled PDF
|-- .gitignore
|-- README.md            # explains the repo and how it deploys

```

Two deliberate points. First, the CV’s `.tex` source sits beside the PDF, so the CV is reproducible from source and verifiably LaTeX-authored rather than exported from a word processor. Second, the featured project lives in its own repository (`Python-AI`); the `projects/` folder holds a case-study write-up that links to that canonical source rather than a second copy of the code — see the design decisions below.

4. Design decisions

Each decision is given with the alternative I rejected and the cost I accepted.

4.1 Hand-written HTML/CSS over a static-site generator

I wrote the page directly rather than using Jekyll or Hugo (both first-class on GitHub Pages) or a React build. For a single page, a generator’s templating, Markdown pipeline, and theme system are overhead that buys nothing, while adding a build step and a dependency tree to maintain. *Cost:* if the site grows to many pages, I will have no templating or content/markup separation and will likely have to refactor toward a generator. For the current scope that trade is clearly worth it, and it keeps the deployment a zero-configuration file copy.

4.2 A user-pages repository for a root-level URL

Naming the repository `PersianSpock.github.io` makes GitHub Pages serve it from the user-level domain root (`persianspock.github.io`). The rejected alternative — an ordinary project repository — would deploy to a `/repo-name/` sub-path, which complicates root-relative links and reads less cleanly. *Cost:* only one site can occupy the user-pages slot, so this “spends” that slot on the portfolio.

4.3 IBM Plex web fonts over the system font stack

Typography is the page’s main source of personality, so I used the IBM Plex superfamily — Serif for display, Sans for body, Mono for data and labels. It is an engineered, system-designed family that suits a data portfolio and avoids the generic high-contrast-serif look. The rejected alternative was the native system font stack, which costs zero requests and never flashes. *Cost:* web fonts add external requests on the render path. I mitigated this with `preconnect` hints and `font-display: swap` (text paints immediately in a fallback, then swaps), and I restricted the request to only the weights actually used. This is the single biggest performance trade on the page and I made it consciously — the empirical section below quantifies it.

4.4 One in-depth project over several shallow ones

The portfolio features a single project in depth rather than a grid of small ones. A real, fully documented pipeline demonstrates more than several toy demos. *Cost*: less apparent breadth, and the portfolio’s strength is tied to one case study; the structure leaves room to add more later.

4.5 Linking to the project’s source over vendoring it

The case-study page links to the canonical `Python-AI` repository and the Colab notebook rather than copying the code into this repository. Copying would create two histories that drift apart. *Cost*: a reader must follow a link to read the code, and the portfolio depends on the external repository remaining public. I judged a single source of truth to be worth more than self-containment, and I attribute every reported metric to the notebook rather than restating it as my own fresh measurement.

4.6 A single stylesheet with design tokens

All styling is one file using CSS custom properties for colour and type. The rejected alternative — utility classes (e.g. Tailwind) or multiple stylesheets — adds tooling or fragmentation that a page this size does not need. *Cost*: no utility ergonomics; discipline is manual.

4.7 Empirical check: page weight and accessibility

The brief rewards decisions justified by experiment, so I measured two things.

Transfer size and requests. The entire render-critical payload — `index.html`, `style.css`, and the SVG favicon — is **20.5 KB raw** / **about 6.5 KB gzipped**, served over **three** local requests. The CV PDF (92 KB) loads only on an explicit download click. The only other first-load cost is the IBM Plex font request, which confirms the quantitative shape of the font trade above: nearly the whole “weight” of the page is its fonts, not its markup or styles.

Colour contrast (WCAG 2.1). I computed contrast ratios for the key text/background pairs. All pass the AA threshold for normal text (4.5:1):

Pair	Ratio	WCAG AA (normal)
Body ink on paper	14.9:1	Pass
Muted text on paper	5.75:1	Pass
Accent (links) on paper	8.56:1	Pass
Footer text on ink	10.8:1	Pass
Turquoise links on ink	5.20:1	Pass

These are computed figures, not estimates. The natural next measurement is a Lighthouse audit on the live URL, which I recommend running with:

```
npx lighthouse https://persianspock.github.io --only-categories=performance,accessibility
```

5. Tools and technologies

- **HTML5** — semantic structure (landmark elements, a skip link).
- **CSS3** — custom properties (design tokens), Grid and Flexbox layout, media queries, keyframe animation, and a `prefers-reduced-motion` fallback.
- **SVG** — the animated ROC curve and the favicon, both vector and tiny.

- **IBM Plex** (Serif / Sans / Mono) via Google Fonts.
- **Git & GitHub** — version control with an incremental commit history.
- **GitHub Pages** — static hosting; pushing to `main` redeploys.
- **LaTeX** (`pdflatex`) — the CV and this report.

6. Critical reflection

Strengths.

- *Lightweight and fast by construction.* At roughly 6.5 KB gzipped of markup and styles over three requests, the page has almost nothing to optimise away.
- *Accessible foundations.* Semantic landmarks, a keyboard skip link, visible focus styles, a reduced-motion fallback, and verified AA contrast.
- *Reproducible and honest artefacts.* The CV is LaTeX from source; the project's numbers are attributed to their notebook rather than re-asserted.
- *A signature with meaning.* The ROC curve is not decoration — it encodes the site's theme and connects to the featured project.

Weaknesses.

- *No automated testing or CI.* There are no HTML/CSS validation or link-checking steps in a pipeline; checks were run manually during the build. A broken link could ship unnoticed.
- *Web fonts on the render path.* Despite `swap` and `preconnect`, the fonts are third-party requests and can cause a brief flash of fallback text; they are the page's main performance cost.
- *Single point of depth.* One featured project means limited breadth, and the portfolio leans on one external repository staying available.
- *Metrics not re-run here.* The project's results are reported from Colab and not regenerated in this build, so they are attributed rather than independently reproduced.
- *Thin discoverability.* Basic meta tags only — no Open Graph tags, sitemap, or structured data.
- *Accessibility not formally audited.* Contrast is verified, but no automated audit (e.g. axe or Lighthouse) or screen-reader pass has been run end to end.

Future improvements.

- Add a GitHub Actions workflow that validates HTML, checks links, and runs a Lighthouse budget on every push — turning the manual checks into enforced ones.
- Self-host subset `woff2` fonts to cut external requests and remove the fallback flash entirely.
- Add Open Graph / Twitter card meta and a sitemap for sharing and SEO.
- Grow the `projects/` section as more public work becomes available, and add a small reproducible script that regenerates a headline metric so at least one figure is verified in-repo.
- Run an automated accessibility audit and a screen-reader pass, and add a finer responsive breakpoint for the smallest viewports.

7. References

Mozilla (2024) *HTML elements reference*. Available at: <https://developer.mozilla.org/en-US/docs/Web/HTML/Element> (Accessed: 24 June 2026).

Mozilla (2024) *Using CSS custom properties (variables)*. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties (Accessed: 24 June 2026).

GitHub (2024) *Configuring a publishing source for your GitHub Pages site*. Available at: <https://docs.github.com/en/pages> (Accessed: 24 June 2026).

IBM (2021) *IBM Plex typeface*. Available at: <https://www.ibm.com/plex/> (Accessed: 24 June 2026).

Google (2024) *Optimize WebFont loading and rendering*. Available at: <https://web.dev/articles/font-best-practices> (Accessed: 24 June 2026).

W3C (2023) *Web Content Accessibility Guidelines (WCAG) 2.1*. Available at: <https://www.w3.org/TR/WCAG21/> (Accessed: 24 June 2026).

Google (2024) *Lighthouse: performance and accessibility auditing*. Available at: <https://developer.chrome.com/docs/lighthouse/overview> (Accessed: 24 June 2026).

IBM (2019) *Telco customer churn sample dataset*. Available at: <https://www.kaggle.com/datasets/blastchar/telco-customer-churn> (Accessed: 24 June 2026).